

SQL (Structured Query Language)

➤ Operations on Table :-

1) INSERT

- It is used to insert a single or a multiple records in a table.



Without specifying column names

Syntax

INSERT INTO table_name **VALUES** (value1, value2, value3....);

Eg. **INSERT INTO** student **VALUES** (101,'abcd');

By specifying column names

Syntax

INSERT INTO table_name (column_name1, column_name2)
VALUES (val1,val2,...);

Eg. **INSERT INTO** student (id, name) **VALUES** (101,'abcd');

2) UPDATE

- **SQL UPDATE** statement is used to change the data of the records held by tables.
- Which rows is to be update, it is decided by a condition.

Syntax

UPDATE table_name **SET** [column_name1= value1,... column_nameN = valueN] [**WHERE** condition]

Eg. **UPDATE** student **SET** name='Abc', address='Pune' **WHERE** id=1

3) DELETE

Remove All Rows

Syntax

DELETE FROM table_name;

Eg. **DELETE FROM** student;

Remove All Rows With Condition

Syntax

DELETE FROM table_name **WHERE** condition;

Eg. **DELETE FROM** student **WHERE** id=101;

➤ JOINS in SQL :-

- A join in SQL is used to combine rows from two or more tables.
- It is performed based on a related column between the tables.
- Joins help to retrieve data stored in multiple tables in a single query.
- They use a matching condition, usually a primary key and foreign key relationship.

➤ Types of JOIN :-

- SQL offers several types of joins to combine rows from two or more tables based on related columns between them.
- The main types of joins are:

Types of JOIN

Inner Join

Left Outer Join

Right Outer Join

Full Outer Join

Cross Join

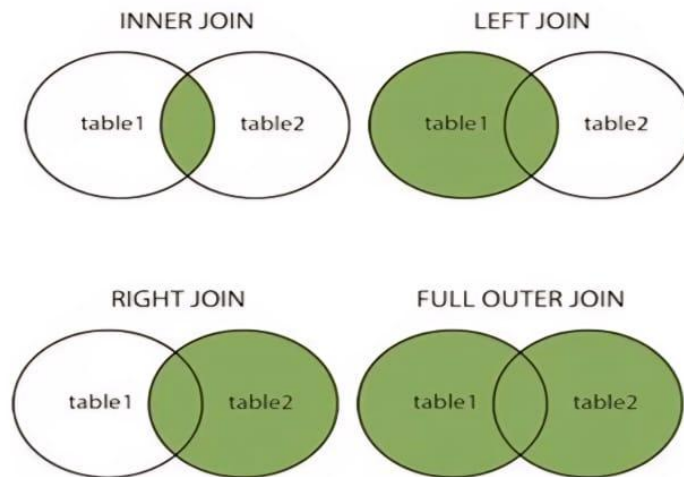


Fig .Diagrammatically Representation of JOINS

Id	Name	Address		Id	Contact	Pincode
1	Abc	Pune	+	1	9874563210	132456
2	Xyz	Mumbai		3	9874563210	789456

Id	Name	Address	Contact	Pincode
1	Abc	Pune	9874563210	132456

Example of JOINS

1) INNER JOIN

Inner Join

The **INNER JOIN** keyword selects records that have matching values in both tables.

Syntax

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Eg. **SELECT * FROM** student1 **INNER JOIN** student2 **ON** student1.id=student2.id;

2) LEFT OUTER JOIN

Left Outer Join

- The SQL left join returns all the values from the left table and it also includes matching values from right table
- if there are no matching join value it returns NULL.

Syntax

```
SELECT column_name(s)  
FROM table1  
LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

Eg. **SELECT * FROM** student1 **LEFT JOIN** student2 **ON** student1.id=student2.id;

3) RIGHT OUTER JOIN

Right Outer Join

- The SQL right join returns all the values from the rows of right table.
- It also includes the matched values from left table
- if there is no matching in both tables, it returns NULL.

Syntax

```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

Eg. **SELECT * FROM** student1 **RIGHT JOIN** student2 **ON** student1.id=student2.id;

4) FULL OUTER JOIN

Right Outer Join

- The SQL right join returns all the values from the rows of right table.
- It also includes the matched values from left table
- if there is no matching in both tables, it returns NULL.

Syntax

```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

Eg. **SELECT * FROM** student1 **RIGHT JOIN** student2 **ON** student1.id=student2.id;

5) CROSS JOIN

Cross Join

- Each row of first table is combined with each row from the second table, known as Cartesian join or cross join.

1st Syntax

```
SELECT * FROM table1 CROSS JOIN table2;
```

Eg. **SELECT * FROM** student1 **CROSS JOIN** student2;

2nd Syntax

```
SELECT * FROM table1, table2;
```

Eg. **SELECT * FROM** student1, student2;